

XM · CLASE 1



Soluciones de Analítica con Databricks

Clase 1: Databricks y el Lakehouse para el sector eléctrico

Docente: Manuela Larrea Gómez

Agenda de la clase 1



01

40 MIN

Teoría: qué es Databricks y por qué un Lakehouse

XM y sus datos · del científico de datos al equipo que opera modelos · Data Warehouse → Data Lake → Lakehouse · Delta Lake · mapa de la plataforma · serverless y Free Edition · casos del sector

02

70 MIN

Práctica: Lab 1 — primer activo gobernado

Acceso al workspace · fork del repositorio y Git folder · cargar los datos de demanda a un volumen · primera tabla Delta · historial y time travel · primer commit

03

10 MIN

Cierre

Solución de referencia · así sería en Azure Databricks · qué viene en la clase 2

XM opera el sistema y administra el mercado: todo eso son datos con trazabilidad

El reto del equipo de analítica no es conseguir datos. Es convertir un análisis descriptivo en un modelo que opera todos los días, gobernado y auditable.



Operación del SIN

Despacho, generación, demanda en tiempo real, restricciones de red. Series de alta frecuencia por agente, área y recurso.

145 mil

registros



Administración del MEM

Transacciones, liquidación y facturación del mercado mayorista. Cifras que se publican, se corrigen y se vuelven a publicar.

355

series activas



SIMEM y datos abiertos

Plataforma de información del mercado: el mismo dato lo consumen agentes, reguladores, academia y el propio XM.

23

fechas de publicación

206

días (ene-jul 2026)

El objetivo del curso: pasar de entrenar modelos a operarlos

Un equipo de ciencia de datos que además sabe gobernar, desplegar y monitorear lo que construye

Hoy (típico en equipos que nacen)

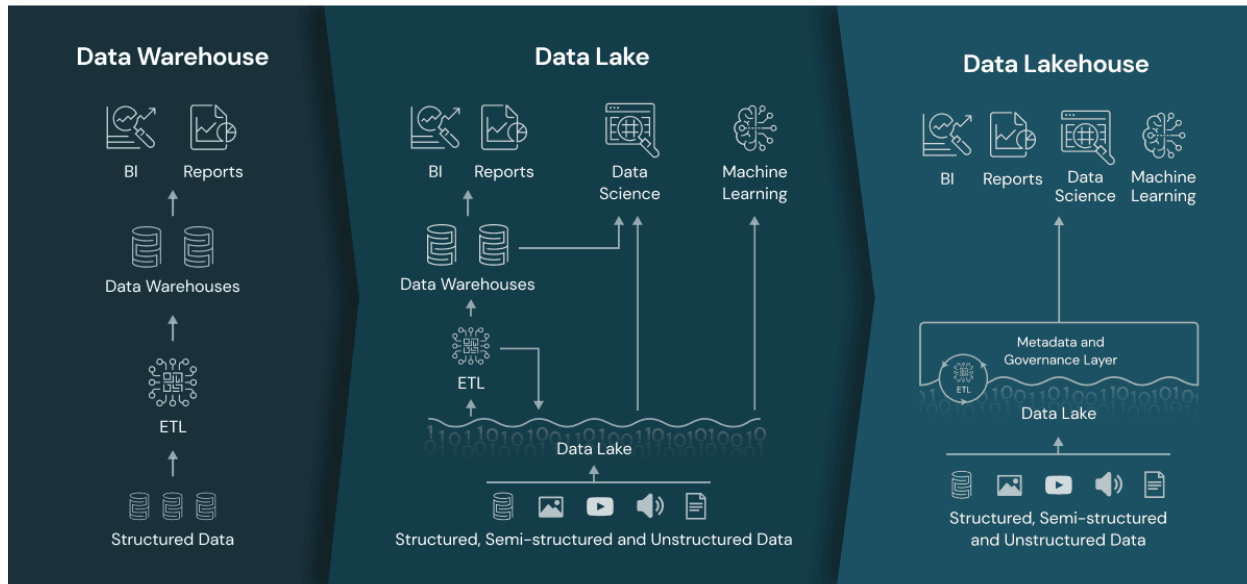
- Datos que llegan limpios por correo o carpeta compartida
- Modelo entrenado en un notebook o en el portátil
- Versión del modelo = fecha del archivo
- Entrega: un Excel o un PowerPoint con los resultados
- Si el modelo se degrada, nadie lo nota hasta que negocio pregunta

Al terminar el curso

- Ingesta gobernada en Bronze → Silver → Gold con reglas de calidad
- Código en un repositorio con tests y revisión por pares
- Modelo registrado en Unity Catalog con versión, firma y alias
- Job programado que ingesta, infiere y publica en Gold; app para consultar
- Monitor de drift, política de reentrenamiento y runbook

Seis hitos: H1 pregunta analítica · H2 arquitectura y fuentes · H3 gobierno y calidad · H4 pipeline y Gold · H5 modelo y despliegue · H6 operación y CI/CD

Cada generación de arquitectura de datos resolvió un problema y creó otro



Cada generación de arquitectura de datos resolvió un problema y creó otro



Data Warehouse

1990s

Estructurado, SQL, reportes. Fuente única de verdad.

Limitación: Rígido y costoso de escalar. No soporta series de medidores a alta frecuencia, archivos ni cargas de ML.



Data Lake

2010s

Barato y flexible. Cualquier formato, cualquier volumen.

Limitación: Sin transacciones ACID: una republicación del mercado a medio escribir deja datos corruptos. Sin gobierno: el pantano de datos.



Lakehouse

hoy

Formato abierto (Delta) sobre almacenamiento de objetos, con ACID, gobierno y ML en el mismo lugar.

Ventaja: Una sola copia del dato para BI, ingeniería y modelos. Es la base de este curso.

El dilema actual: warehouse y lake en paralelo, datos duplicados y dos gobiernos distintos. El Lakehouse los une.

El Lakehouse combina flexibilidad, rendimiento y gobierno en una sola plataforma



Almacenamiento abierto

Objetos en la nube (ADLS, S3) en formatos abiertos: Parquet y Delta. Economía y escala del data lake, sin encierro de proveedor.



Rendimiento y ACID

Transacciones, control de concurrencia y velocidad de consulta de warehouse sobre archivos.



Todos los datos

Tablas, JSON, series de medidores, documentos, imágenes de inspección de activos.



Una única copia

El mismo dato sirve BI, ingeniería, modelos y agentes. Sin extracciones paralelas.



Gobierno unificado

Permisos, linaje y auditoría en un solo lugar, desde el archivo crudo hasta el modelo.

Data Warehouse + Data Lake = Lakehouse

Delta Lake es el formato que hace posible el Lakehouse

Parquet + un log de transacciones. Cuatro garantías que usaremos desde hoy



Transacciones ACID

Una escritura se ve completa o no se ve. Dos jobs no se pisan.



Time travel

Cada escritura es una versión. Consultar la tabla como estaba el día que se publicó un pronóstico.



Esquema controlado

Rechaza columnas inesperadas o las incorpora de forma explícita. Nada entra en silencio.



MERGE idempotente

Insertar, actualizar o ignorar según llave. Base para manejar repeticiones.

```
delta_basics.sql

-- ¿Qué cambió y cuándo?
DESCRIBE HISTORY bronze_energia.demanda_raw;

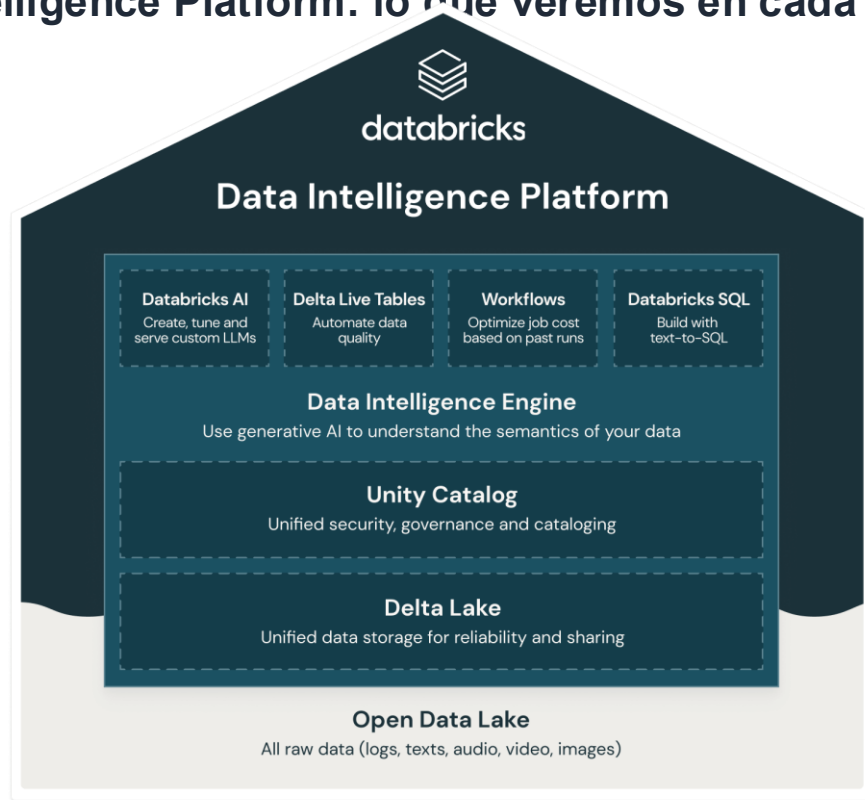
-- La tabla tal como estaba en la versión 3
SELECT count(*) FROM bronze_energia.demanda_raw
  VERSION AS OF 3;

-- O como estaba en una fecha
SELECT * FROM bronze_energia.demanda_raw
  TIMESTAMP AS OF '2026-07-10';

-- Volver atrás si algo salió mal
RESTORE TABLE bronze_energia.demanda_raw
  TO VERSION AS OF 3;
```

En el mercado la misma fecha se publica varias veces (FechaPublicacion $\neq$ Fecha). Delta permite conservar todas las versiones.

Mapa de la Data Intelligence Platform: lo que veremos en cada clase



Mapa de la Data Intelligence Platform: lo que veremos en cada clase

Todo comparte el mismo dato y el mismo gobierno



Unity Catalog

Catálogos, permisos, linaje, auditoría, volúmenes, modelos

Clase 3



Lakeflow

Connect (ingesta), Declarative Pipelines (transformación), Jobs (orquestación)

Clases 4, 5, 12



SQL y AI/BI

Warehouses serverless, editor SQL, dashboards, Genie

Clases 6, 11



Mosaic AI

MLflow, Feature Store, Model Registry, Model Serving, Agent Bricks

Clases 7, 9–11



Databricks Apps

Aplicaciones web (Streamlit, Dash) gobernadas por UC

Clase 11



Lakebase

Base transaccional Postgres integrada al Lakehouse

Clase 11



Asset Bundles

Despliegue declarativo de jobs, pipelines y modelos por ambiente

Clases 13, 14



Delta Sharing y Marketplace

Compartir datos con otros agentes sin copiarlos

Clase 3

Serverless: el cómputo deja de ser un problema del científico de datos

En Free Edition solo existe serverless. En Azure Databricks es la opción recomendada para la mayoría de cargas

	Clúster clásico	Serverless
Quién lo configura	El usuario: tipo de nodo, tamaño, runtime, autoescalado	Databricks. El usuario solo ejecuta
Arranque	Minutos	Segundos
Costo	Se paga mientras está encendido, esté en uso o no	Se paga por lo que se ejecuta
Gobierno	Políticas de clúster para controlar lo que cada equipo puede crear	Políticas más simples; presupuestos por workspace
Cuándo usarlo	Librerías nativas especiales, GPU, control fino del entorno	Notebooks, jobs, pipelines, SQL: el caso general

Práctica de esta clase: no crearán ningún clúster. Abrir un notebook y ejecutar es todo lo que hay que hacer.

Databricks Free Edition: qué tiene, qué no, y cómo lo cubrimos

Uso no comercial. Es nuestro laboratorio, no un ambiente productivo

Capacidad	En Free Edition 	En el curso
Unity Catalog, volúmenes, linaje	Sí: un workspace y un metastore	Catálogos dev / qa / prod simulan ambientes
Cómputo	Solo serverless; un SQL warehouse 2X-Small	Todo el curso corre serverless
Lakeflow Declarative Pipelines	Un pipeline activo por tipo	Un pipeline Bronze → Silver
Jobs	Máximo 5 tareas concurrentes	DAG diario de 5 tareas
MLflow y Model Registry	Sí	Núcleo de las clases 9–11
Model Serving	CPU, endpoints limitados	Un endpoint para el modelo campeón
Databricks Apps	Hasta 3 apps; corren 24 h tras desplegar	Una app de consumo
Azure (ADLS, Key Vault, DevOps)	No	Se diseña y documenta; CI/CD con GitHub Actions

Operadores de red y de mercado ya operan así: tres referencias

Lo que hacen sobre Databricks y lo que de eso aplica a XM

AEMO · Australia

Operador del mercado y del sistema eléctrico australiano. Adoptó la plataforma para unificar datos de operación y mercado bajo un mismo gobierno.



Rol equivalente al de XM: operador de sistema + administrador de mercado.

EDP E-REDES · Portugal

Operador de distribución. Pronostica ~200.000 curvas de carga cada día (intervalos de 15 min) con Spark, pandas UDFs y Workflows; ensamble de LightGBM, Prophet y baselines.



Patrón de modelo global sobre muchas series: el mismo que usaremos con 355 series.

Octopus Energy · Reino Unido

Comercializador con plataforma de datos sobre Databricks para pronóstico, tarifas y personalización a escala.



Pronóstico como servicio interno consumido por varias áreas.

Fuentes: Databricks, Data + AI Platform for Energy; Databricks Community, EDP E-REDES load forecasting. Referencias completas en lecturas.md de la clase.

Nuestro caso: demanda y pérdidas de energía por segmento

El hilo conductor de las 15 clases

Para cada combinación activa de operador de red, mercado de comercialización, tipo de mercado y sector CIU, ¿cuál es la demanda de energía esperada para los próximos 7 días?

Columna	Qué es
FechaPublicacion	Cuándo se publicó el registro
Fecha	Día al que corresponde el valor
CodigoSICAgente	Operador de red (27)
TipoMercado	Regulado / No Regulado
MercadoComercializacion	Mercado (29)
ClasificacionIndustrial	Sección CIU (22); Regulado = SIN CLASIFICAR
Valor / CodigoVariable	kWh; DdaReal o PerdidasEnergia



Lo que importa

No el algoritmo. Que el modelo se ingeste, gobieme, versione, despliegue y monitoree como un producto.



Lo que ya sabemos del dato

Formato largo (dos filas por serie-día). Siete meses: corto para pronóstico; en la clase 7 sumamos fuentes.

Cómo trabajaremos: un repositorio que crece clase a clase

github.com/<usuario>/Databricks-to-XM — cada participante trabaja en su fork

```
Databricks-to-XM/
├─ README.md           plan de las 15 clases
├─ clases/c01...c15/   slides, lab, notebooks
├─ databricks.yml      bundle dev / qa / prod
├─ resources/          pipeline y jobs
├─ conf/               config por ambiente
├─ src/xm_demanda/     lógica en Python
├─ notebooks/          delgados, uno por tarea
├─ tests/              pytest
├─ data/raw/           DemandaPerdidas.xlsx
├─ docs/               arquitectura, ADRs, autoevaluación
└─ .github/workflows  CI/CD
```



Toda clase termina en commit

Sin commit no hay hito. Rama feature/c01-<tema>, PR hacia main de tu fork; CI (lint y tests) es el revisor.



Decisiones por escrito

Una ADR corta por decisión de arquitectura. Es lo que se llevan a XM.



Trabajo autónomo

Cada lab termina con verificar(): te dice qué falta. Al cierre se libera el tag sXX de la solución para comparar con git diff.



Autoevaluación por hito

docs/autoevaluacion.md: sé que terminé cuando... Niveles funcional, sólido y operable, si quieres ir más allá.

Notebooks delgados, lógica en src/, nada hard-codeado, secretos fuera del repo. Desde hoy.

Laboratorio 1: primer activo gobernado en el Lakehouse (1/2)

Objetivo: dejar los datos de demanda como tabla Delta en Unity Catalog, desde el repositorio, con historial

1

Entrar al workspace

Acepta la invitación al workspace compartido del curso. Autenticación por correo (código OTP), Google o Microsoft. Verifica que ves el catálogo workspace en Catalog.

2

Fork y Git folder

Haz fork de Databricks-to-XM en tu cuenta de GitHub. En Databricks: Workspace → Create → Git folder, pega la URL de tu fork. Crea la rama feature/<equipo>-c01-primer-tabla.

3

Esquema y volumen

En un notebook SQL: `CREATE SCHEMA workspace.c01_<usuario>; CREATE VOLUME workspace.c01_<usuario>.raw`. Sube data/raw/DemandaPerdidas.xlsx al volumen desde Catalog Explorer.

4

Abrir el notebook del lab

clases/c01-databricks-lakehouse/notebooks/lab01_primer_tabla_delta. Completa el widget usuario y ejecuta celda por celda. No hay dúster que crear: es serverless.

Laboratorio 1: primer activo gobernado en el Lakehouse (2/2)

5

Crear la tabla Delta

Lee el Excel con pandas, conviértelo a Spark, agrega `_ingested_at` y `_source_file` y escribe `workspace.c01_<usuario>.demanda_raw`.

6

Consultar en SQL y en Python

Cuenta filas por `CodigoVariable`. Confirma 145.408 filas y 355 series activas.

7

Historial y time travel

Vuelve a ejecutar la escritura. `DESCRIBE HISTORY`: dos versiones. Consulta `VERSION AS OF 0`.

8

Commit y push

Desde el Git folder: `commit "feat(c01): primera tabla Delta de demanda"` y push a tu fork.

```
lab01 - fragmento

import pandas as pd
from pyspark.sql import functions as F

pdf = pd.read_excel(f"{vol}/{archivo}")
df = (spark.createDataFrame(pdf)
      .withColumn("_ingested_at",
                  F.current_timestamp())
      .withColumn("_source_file",
                  F.lit(archivo)))
df.write.mode("overwrite")
      .saveAsTable(f"{schema}.demanda_raw")
```

Criterios de aceptación

- Tabla `demanda_raw` con 145.408 filas y columnas `_ingested_at` y `_source_file`
- `DESCRIBE HISTORY` muestra al menos 2 versiones
- Commit visible en tu fork en GitHub

Extensión opcional: ¿cuántas series tienen datos todos los 206 días? ¿Cuáles no? Guarda la consulta en el notebook.

Así sería en Azure Databricks: lo que hoy simulamos y lo que cambia en producción

Elemento	Hoy en Free Edition	En XM sobre Azure Databricks
Workspace	Uno compartido para todos	Uno por ambiente (dev, qa, prod), en la suscripción y red de XM
Identidad	Correo, Google o Microsoft	Microsoft Entra ID con SCIM: grupos corporativos sincronizados a Unity Catalog
Almacenamiento	Gestionado por Databricks	ADLS Gen2 propio, external locations y storage credentials gobernados por UC
Secretos	Secret scopes de Databricks	Secret scopes respaldados por Azure Key Vault
Cómputo	Serverless	Serverless por defecto; clústeres con políticas cuando se necesite control fino
Red	Internet, dominios confiables	Private Link, sin IP pública, egress controlado
Código y despliegue	GitHub + GitHub Actions	Azure Repos o GitHub + Azure DevOps Pipelines; mismos Asset Bundles

Lo que no cambia: el código, el bundle, los notebooks, las reglas de calidad y el modelo. Eso es lo que construimos en el curso.

Resumen de la clase 1 y preparación para la clase 2

Logros de hoy

✓ Lakehouse

Por qué reemplaza al warehouse y al lake en paralelo

✓ Delta Lake

ACID, time travel, esquema, MERGE

✓ Plataforma

Mapa de capacidades y en qué clase se ve cada una

✓ Free Edition

Qué permite, qué simulamos, qué documentamos

✓ Lab 1

Fork, Git folder, volumen, primera tabla Delta, commit

Próxima clase

→ Medallion aplicado al caso

Contratos por capa: esquema, dueño, frescura

→ Diseño en equipo

docs/architecture.md y ADR-001 (granularidad y horizonte)

→ Caso: EDP E-REDES

Qué de su patrón aplica a 355 series y qué no



Gracias

Nos vemos en la clase 2: Arquitectura Medallion y patrones analíticos

DOCENTE

Manuela Larrea Gómez